

Interview Preparation Playbook — Abubaker Siddique

Java Backend Engineer · 5.5 YOE · Bengaluru Target companies: Walmart, Target, Samsung, Akamai (product-based MNCs, ~18-40 LPA band)

How to Use This Playbook

This is your single source of truth for the next 4 weeks. Work through it in order, but revisit Sections 3-7 repeatedly. Check off items as you go.

Reality check (read this first): You have a strong, honest backend foundation — billing systems at real scale, Kafka, Spring Boot, JWT, cloud basics. Your two biggest gaps for this tier are **(1) System Design fluency** and **(2) DSA pattern recognition under time pressure**. This playbook weights those two areas the most. Core Java you mostly know — we'll sharpen the tricky edges.

Your honest starting scorecard:

Area	Current Level	Target	Priority
Core Java	6.5/10	8.5/10	Medium
Spring Boot / Microservices	7/10	8.5/10	Medium
System Design	4/10	7.5/10	● Highest
DSA / Coding	4.5/10	7/10	● Highest
Behavioral / STAR	5/10	8/10	High
DB / DevOps / Cloud	6/10	7.5/10	Medium

1. Resume & Pitch Optimization

1.1 Quick Strengths / Weaknesses Analysis

Strengths your resume already signals:

- Real scale** — \$30M revenue, ~7,000 enterprise customers, millions of SKUs. This is your single biggest asset; lead with it.
- Breadth across domains** — telecom billing, fintech (MCA), ITSM, retail. Shows adaptability.

- **Honest ownership gradient** — "Stabilized/Owned" vs "Worked on" reads as credible, not inflated.
- **Production support depth** — you've fixed real prod issues (Bill Rerun, query timeout, log injection). Product companies value this.
- **Event-driven exposure** — Kafka + Avro is directly relevant to Walmart/Target scale systems.

Weaknesses to pre-empt (interviewers will probe these):

Gap	Risk	Quick-Win Mitigation
No system design ownership	"Have you designed anything from scratch?"	Prepare 2 stories where you made design <i>decisions</i> (Bill Rerun orchestration, MCA API structure)
Java 8/11 only (no 17/21)	Modern-stack roles may probe records, sealed classes, virtual threads	Spend 2-3 hrs learning Java 17/21 headline features (Section 3) — enough to discuss
Mostly monolith + small services	"Largest scale you've handled?"	Frame honestly: "~7,000 customers, batch chunks of 1,000" + show you understand how to scale beyond
Consultant/service-firm background	Bias toward product-company "engineering rigor"	Emphasize testing (JUnit/Mockito/JaCoCo), CI/CD, code review discipline
Frontend was AI-generated	Don't get caught claiming React depth	Stick to "backend refactor" — already handled in resume

1.2 "Tell Me About Yourself" — Master Script (90 seconds)

Delivery tips: Speak in 3 beats — *Who I am* → *What I've done* → *What I'm looking for*. Practice out loud until it's 75-90 seconds. Don't recite; internalize the flow.

Master version:

"I'm a Java backend engineer with about five and a half years of experience, mostly building and supporting enterprise-scale systems. For the last two and a half years I've been at Lumen Technologies, working on MBS — a telecom billing platform that serves around 7,000 enterprise customers and supports over \$30 million in annual revenue. My strongest work there was stabilizing the Bill Rerun workflow — it was failing and forcing billing agents to manually intervene multiple times a week. I refactored the orchestration logic and added structured success, partial, and failure handling, which eliminated that manual effort. I also contributed to a larger ERP transformation, where

we consolidated three upstream input services into a single SAP-BRIM source, and I built the inbound APIs for data mapping and persistence.

Before Lumen, I worked across fintech and retail domains — I built a Merchant Cash Advance underwriting engine with JWT-secured APIs, and earlier at TCS I worked on a Kafka-based event-driven platform for retail inventory, processing millions of SKUs. My core stack is Java, Spring Boot, REST, JPA, and Kafka. I'm now looking to move into a product company like yours where I can work on backend systems at larger scale and grow into more system-level ownership."

Why this works: Opens with scale, leads with your best story, shows breadth, ends with a forward-looking, company-aligned close.

Variations by Round Type

For a hiring manager (emphasize impact + ownership): Trim the TCS detail, expand on Bill Rerun outcome and ERP contribution. End on "I want more end-to-end ownership."

For a peer/technical interviewer (emphasize stack + depth): Lead with the stack, mention specific technical decisions (JPQL refactor, JWT stateless design, Kafka producer/consumer). Shorter on business framing.

For a 30-second elevator version:

"Java backend engineer, 5.5 years. Currently at Lumen on a telecom billing platform serving 7,000 enterprise customers. Strongest in Spring Boot, REST APIs, JPA, and Kafka. I own production-critical billing workflows and I'm looking to move into a product company at larger scale."

1.3 Anticipated Follow-Up Questions (Resume-Driven)

Prepare crisp answers for each. These *will* come up:

#	Likely Question	What They're Testing
1	"Walk me through the Bill Rerun problem end to end."	Depth of ownership, debugging approach
2	"What exactly was failing before your fix?"	Root-cause thinking
3	"How did the ERP/BRIM integration handle data mapping failures?"	Error handling, resilience
4	"The query was timing out — how did you diagnose it?"	Performance debugging, SQL/JPA depth
5	"Why JWT over session-based auth in the MCA platform?"	Auth design trade-offs
6	"How did you handle Kafka consumer failures / retries?"	Distributed systems thinking
7	"What was the largest scale you personally handled?"	Honesty + scale awareness
8	"What would you have done differently on MBS?"	Reflection, growth mindset
9	"You used Copilot — how do you ensure code quality?"	Engineering discipline
10	"Why are you leaving / why product companies now?"	Motivation, retention risk

Sample answer — Q4 (query timeout diagnosis):

"The Bill Preview page was timing out for agents pulling high-volume invoice data. I started by reproducing it with a representative customer, then looked at the JPQL being generated. The query was doing inefficient joins and pulling more than it needed. I refactored the query to filter earlier and return only the required projection, which brought the load time back to acceptable. If I had more time, I'd have added an index review and pagination as well."

Note the honest close — naming what you'd *also* do shows senior thinking without claiming you did it.

Sample answer — Q10 (why leaving):

"My time at Lumen was valuable — I got real ownership of production-critical billing flows. The role is wrapping up, and I want my next move to be a product company where backend systems run at larger scale and I can grow into more system-level design ownership. Companies like yours are exactly that environment."

2. Behavioral & Leadership (STAR Stories)

2.1 The STAR Framework (Calibrated for You)

Situation (1 sentence) → Task (1 sentence) → Action (3-4 sentences, *this is the meat*) → Result (1-2 sentences, *quantify*).

Your most common mistake will be: spending too long on Situation and too little on Action. Interviewers want *what you specifically did*. Use "I" not "we" when describing your actions.

2.2 Your 3 Anchor Stories (Strengthened)

★ Story 1 — Bill Rerun Stabilization (Production Problem Solving)

Element	Content
S	At Lumen, the MBS billing platform's Bill Rerun workflow was unreliable — when a rerun partially failed, the system didn't clearly report what succeeded vs failed.
T	Billing agents were manually intervening multiple times a week to clean up inconsistent states. I was asked to make the workflow reliable.
A	I traced the orchestration logic and found it lacked structured handling of partial outcomes. I refactored it to explicitly distinguish success, partial-success, and failure states, and made the response contract clear so downstream consumers and agents knew exactly what happened. I added consistency checks so a failed rerun wouldn't leave half-written data.
R	This eliminated the recurring multiple-times-weekly manual interventions and made the workflow trustworthy for the agents operating a platform serving ~7,000 customers.

Maps to interview prompts: "production issue," "ownership," "reliability," "improving a fragile system."

★ Story 2 — ERP / BRIM Transformation (Cross-Team Initiative)

Element	Content
S	MBS was ingesting billing input from 3 separate upstream services, which created inconsistency and maintenance overhead.
T	The team undertook an ERP transformation to consolidate these into a single SAP-BRIM source of truth. I owned the inbound API layer.
A	I developed the inbound APIs that consumed data from SAP, mapped it to MBS billing database fields, and persisted it. I had to carefully handle field-level mapping

Element	Content
	differences between SAP's model and ours, and coordinate with the team on data contracts.
R	The migration went live with minimal defects, and the consolidation reduced the number of integration points the team had to maintain. The team was recognized with the Lumen India Team Highlight Award.

Maps to: "cross-team work," "significant project," "handling complexity," "migration."

★ **Story 3 — MCA Underwriting Engine (End-to-End Ownership)**

Element	Content
S	At RECV, a fintech platform needed an underwriting calculation engine for Merchant Cash Advances, where repayment is a percentage of merchant sales rather than fixed EMIs.
T	I owned the backend — the calculation engine, the APIs, authentication, and persistence.
A	I designed the dashboard API that computes factor rate, repayment tenor, month-by-month holdback schedules, fee splits, and a sales stress matrix from -10% to -50%. I implemented JWT authentication with Spring Security — BCrypt hashing, stateless sessions, a Bearer token filter — and role-based access. I backed it with PostgreSQL and Spring Data JPA and documented everything with OpenAPI.
R	I delivered a working underwriting backend that lenders could use to model advances, with the calculation logic and auth fully owned by me.

Maps to: "something you built end to end," "technical depth," "ownership."

2.3 Eight Additional High-Impact Stories

For each, I've given you the **skeleton + the angle**. Fill in specifics from memory and rehearse. Keep each to 90 seconds.

Story 4 — Performance: The Query Timeout (Technical Debugging)

- **Angle:** Methodical debugging under user pressure.
- **S/T:** Bill Preview timing out for agents on high-volume data.
- **A:** Reproduced with representative customer → inspected generated JPQL → found inefficient joins/over-fetching → refactored to filter early and project only needed fields.
- **R:** Restored reliable page loads; agents unblocked.
- **Maps to:** "time you improved performance," "debugging a hard problem."

Story 5 — Security: Log Injection Fix (Attention to Detail / Ownership)

- **Angle:** You proactively hardened the system, not just shipped features.
- **S/T:** Customer-facing controllers were vulnerable to log injection (log forging).
- **A:** Identified unsanitized input flowing into logs across GUI controllers → applied input sanitization → verified the fix didn't break legitimate logging.
- **R:** Closed a real security vulnerability in a customer-facing billing portal.
- **Maps to:** "going beyond your assigned work," "security mindset," "attention to detail."

Story 6 — Learning Fast: Telecom Billing Domain (Adaptability)

- **Angle:** Picked up a complex, unfamiliar domain (rating, taxing, invoicing, BRIM).
- **S/T:** Joined MBS with no telecom-billing background; needed to be productive on a multi-step billing pipeline.
- **A:** Studied the invoice lifecycle (rating → billing → taxing → formatting), traced existing code, asked targeted questions, started with smaller defects before owning Bill Rerun.
- **R:** Became a reliable owner of billing workflows within months.
- **Maps to:** "learning something difficult quickly," "ramping up in a new domain."

Story 7 — Working With AI Tools Responsibly (Modern Engineering Judgment)

- **Angle:** You use AI to move fast *but* maintain quality discipline.
- **S/T:** Tight delivery timelines on backend features and tests.
- **A:** Used Copilot (GPT-4, Claude) to generate Spring Boot boilerplate, draft JUnit/Mockito tests, and accelerate refactoring — but reviewed every output, adapted it to our patterns, and never merged code I couldn't explain.
- **R:** Faster delivery without sacrificing maintainability or review standards.
- **Maps to:** "how do you use AI," "staying current," "balancing speed and quality."

Story 8 — Handling Ambiguity: ATC Edit-Flow Refactor (Decision-Making)

- **Angle:** A messy, fragmented process that you simplified.
- **S/T:** The ATC project edit flow used fragmented multi-API operations that caused partial-update errors.
- **A:** Analyzed the existing edit operations → designed a single cohesive Spring Boot API pattern → consolidated the flow to reduce partial-update risk.
- **R:** Fewer errors, more maintainable code.
- **Maps to:** "simplifying something complex," "making a design decision."

Story 9 — A Failure / Mistake (Growth Mindset) ⚠️ *Prepare this carefully*

- **Angle:** Pick a *real, small, recoverable* mistake. Interviewers distrust "I have no failures."
- **Template:** A time a fix you shipped had an unintended side effect, you caught it (or it was caught in test/review), you owned it, fixed it, and added a safeguard (a test, a check) so it couldn't recur.
- **R:** Emphasize the *learning* and the *systemic safeguard* you added.
- **Maps to:** "tell me about a failure," "a time you were wrong."
- **Coaching:** We'll build this together — tell me a real instance and I'll help you frame it safely.

Story 10 — Disagreement / Conflict (Collaboration)

- **Angle:** A technical disagreement resolved through data, not ego.
- **Template:** You disagreed with an approach (e.g., how to handle a billing edge case, or a query approach), you raised it respectfully, brought evidence/reasoning, and you either persuaded the team or disagreed-and-committed.
- **R:** Outcome + preserved relationship.
- **Maps to:** "disagreed with a teammate," "conflict," "influencing without authority."

Story 11 — Mentoring / Knowledge Sharing (Leadership Without Authority)

- **Angle:** Even without being a lead, you shared knowledge.
- **Template:** You documented a tricky part of the billing flow, helped onboard someone, explained the Bill Rerun logic to a teammate, or did a knowledge-transfer session.
- **R:** Made the team less dependent on tribal knowledge.
- **Maps to:** "mentoring," "leadership," "helping others."
- **Note:** Be honest — frame it as knowledge-sharing, not formal mentorship, if that's accurate.

Story 12 — Delivering Under Pressure (Prioritization)

- **Angle:** A tight deadline where you prioritized ruthlessly.
- **Template:** A release or production fix with time pressure (a Bill Rerun incident, a go-live for BRIM), where you focused on the critical path, communicated status, and delivered.
- **R:** Met the deadline / resolved the incident.
- **Maps to:** "tight deadline," "high-pressure situation," "prioritization."

2.4 Company-Value Alignment Cheat Sheet

Company	Known Cultural Emphasis	Lead With These Stories
Walmart	Scale, customer focus, pragmatism, ownership	Bill Rerun (scale + ownership), ERP (large initiative), Kafka (event scale)
Target	Engineering craft, collaboration, reliability	Query timeout (craft), Log injection (rigor), Conflict (collaboration)
Samsung	Technical depth, performance, systems	MCA engine (depth), Query timeout (performance), Kafka (systems)
Akamai	Distributed systems, reliability, performance at edge	Kafka (distributed), Bill Rerun (reliability), Query timeout (performance)

2.5 Behavioral Practice Checklist

- ☐ Record yourself telling all 3 anchor stories (90s each) — listen back
- ☐ Write 1-line S/T/A/R bullets for stories 4–12 on index cards
- ☐ Build Story 9 (failure) with a real example — bring it to next session
- ☐ Build Story 10 (conflict) with a real example
- ☐ Practice the "why leaving" answer until it's smooth and non-negative
- ☐ Rehearse "Tell me about yourself" until 75–90s without notes

3. Core Java & Advanced Topics

Your reality: You work in Java 8/11 daily, so the fundamentals are there. The gaps are: (1) deep concurrency, (2) JVM internals under the hood, (3) Java 17/21 features you haven't used. Prioritize by the ● / ● / ● markers.

3.1 Prioritized Concept Map

Topic	Priority	Why It Matters For You
Collections internals (HashMap, ConcurrentHashMap)	●	Asked in almost every Java interview
Concurrency (threads, executors, locks, atomics)	●	Your weakest core-Java area; heavily tested at 5+ YOE
<code>equals()</code> / <code>hashCode()</code> contract	●	Classic, you must nail this
Exception handling & try-with-resources	●	You use it; know the edge cases

Topic	Priority	Why It Matters For You
Generics, wildcards	●	Comes up in design discussions
Java 8 streams & functional interfaces	●	You use these; know the tricky bits
JVM memory model, GC	●	Senior-level differentiator
Java 17/21 features	●	Learn enough to <i>discuss</i> (records, sealed, virtual threads)
String pool, immutability	●	Quick wins

3.2 Must-Know Tricky Questions + Crisp Answers

Q: How does HashMap work internally? What changed in Java 8?

A HashMap stores entries in an array of buckets, indexed by `hash(key)`. Collisions form a linked list in each bucket. **In Java 8**, when a bucket exceeds 8 entries (and table size ≥ 64), the linked list converts to a balanced **red-black tree**, improving worst-case lookup from $O(n)$ to $O(\log n)$. Load factor default is 0.75; resize doubles capacity and rehashes.

Q: HashMap vs ConcurrentHashMap?

HashMap is not thread-safe — concurrent modification can corrupt it or cause infinite loops (pre-Java 8). ConcurrentHashMap allows concurrent reads and segmented/bucket-level writes. **In Java 8** it dropped segments for a finer-grained approach using CAS + synchronized on bin heads. It never locks the whole map for reads.

Q: Explain the equals/hashCode contract.

If two objects are equal via `equals()`, they **must** have the same `hashCode()`. The reverse isn't required (equal hashcodes don't imply equality — that's a collision). Breaking this means objects get "lost" in HashMaps/HashSets. Always override both together.

Q: `synchronized` vs `ReentrantLock`?

`synchronized` is simpler, JVM-managed, auto-releases on exit/exception. `ReentrantLock` is more flexible: `tryLock()` with timeout, interruptible locking, fairness policy, and multiple condition variables. Use `synchronized` unless you need those features.

Q: What is `volatile`? Does it make `count++` thread-safe?

`volatile` guarantees **visibility** (reads/writes go to main memory, not thread cache) and prevents reordering — but **not atomicity**. `count++` is read-modify-write (3 operations), so `volatile` does NOT make it thread-safe. Use `AtomicInteger` / `incrementAndGet()` instead.

Thread creation is expensive and unbounded thread creation can exhaust resources. `ExecutorService` pools and reuses threads, queues tasks, and decouples submission from execution. Use `Executors.newFixedThreadPool(n)` for bounded concurrency. Always `shutdown()`.

Q: Checked vs unchecked exceptions — when to use which?

Checked (`extends Exception`): recoverable conditions the caller should handle (e.g., `IOException`). Unchecked (`extends RuntimeException`): programming errors (e.g., `NullPointerException`, `IllegalArgumentException`). Modern style (and Spring) leans toward unchecked for cleaner APIs.

Q: What's the difference between `==` and `equals()` for Integers?

`==` compares references. Integer caches values **-128 to 127** (autoboxing via `Integer.valueOf()`), so `Integer a=100, b=100; a==b` is `true`, but `a=200, b=200; a==b` is `false`. Always use `.equals()` for object comparison.

3.3 Java 17/21 Quick-Win Features (Learn to Discuss)

You don't need production experience — just be able to explain these if asked "what's new in modern Java":

```
java
```

```
// Records (Java 16) – immutable data carriers, auto equals/hashCode/toString
record Customer(String id, String name, BigDecimal balance) {}

// Sealed classes (Java 17) – restrict which classes can extend
sealed interface Payment permits CardPayment, BankTransfer {}

// Pattern matching for switch (Java 21)
String describe(Payment p) {
    return switch (p) {
        case CardPayment c -> "Card ending " + c.last4();
        case BankTransfer b -> "Transfer from " + b.account();
    };
}

// Virtual threads (Java 21) – lightweight threads for high-concurrency I/O
// Huge for backend: millions of concurrent blocking tasks cheaply
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    executor.submit(() -> handleRequest());
}

// Text blocks (Java 15)
String json = """
    { "id": 1, "name": "test" }
    """;
```

One-liner you can say: *"I've been on Java 8 and 11 in production, but I've kept up with the newer features — records for DTOs, sealed classes for closed hierarchies, and virtual threads in 21 which are a big deal for high-concurrency I/O-bound backends. I'd be excited to use them."*

3.4 Core Java Practice Checklist

- ☐ Re-implement HashMap put/get logic on paper
- ☐ Write a thread-safe counter 3 ways (synchronized, AtomicInteger, ReentrantLock)
- ☐ Explain happens-before relationship out loud
- ☐ Write a producer-consumer using BlockingQueue
- ☐ Solve 5 stream API problems (grouping, partitioning, flatMap)
- ☐ Be able to discuss all 5 Java 17/21 features above

4. Spring Boot & Microservices

Your reality: This is one of your stronger areas — you've built REST APIs, used JPA/Hibernate, JWT/Spring Security, and worked in a microservices ecosystem. Goal here is to convert *usage* into *articulate depth*.

4.1 Deep-Dive Topics (Ranked)

Topic	Priority	Likely Question
Spring Bean lifecycle & scopes	●	"What happens when Spring starts up?"
Dependency Injection internals	●	"Constructor vs field injection — why?"
<code>@Transactional</code> deep behavior	●	"How does it work? Propagation? Pitfalls?"
REST API design best practices	●	"How do you version APIs? Idempotency?"
Spring Security / JWT flow	●	"Walk me through your JWT auth" (you built this!)
JPA/Hibernate (N+1, lazy/eager, caching)	●	"What's the N+1 problem?"
Microservices comm (sync/async)	●	"REST vs messaging — when?"
Resilience (circuit breaker, retry, timeout)	●	"How do you handle a downstream failure?"
Config & profiles	●	"How do you manage env config?"
Observability (logging, metrics, tracing)	●	"How do you debug across services?"

4.2 High-Value Q&A

Q: Constructor vs field injection — which and why?

Constructor injection is preferred: it makes dependencies explicit, allows `final` fields (immutability), makes the class testable without Spring, and fails fast if a dependency is missing. Field injection (`@Autowired` on a field) hides dependencies and can't be made final. Use constructor injection.

Q: Explain `@Transactional`. What are the gotchas?

It wraps a method in a database transaction via an AOP proxy — commit on success, rollback on a runtime exception. **Gotchas:**

1. **Self-invocation** — calling a `@Transactional` method from another method in the *same class* bypasses the proxy, so the transaction doesn't apply.
2. **By default only rolls back on unchecked exceptions** — checked exceptions need `rollbackFor`.
3. **Propagation matters** — `REQUIRED` (default) joins an existing transaction; `REQUIRES_NEW` suspends and starts a fresh one.

Q: What is the N+1 query problem? How do you fix it?

When you fetch a list of N entities and then lazily access a relationship on each, Hibernate fires 1 query for the list + N queries for the relationships = N+1 queries. **Fixes:** use a `JOIN FETCH` in JPQL, an `@EntityGraph`, or batch fetching (`@BatchSize`). This is *highly* relevant to your billing work — mention you've dealt with query performance.

Q: Walk me through your JWT authentication. (Your RECV story — own this!)

"On user login, I validate credentials (BCrypt-hashed passwords) and issue a signed JWT containing the user identity and roles. The token is stateless — no server-side session. On each request, a Bearer token filter extracts and validates the token signature and expiry, then sets the security context. Spring Security's filter chain then enforces role-based access on protected endpoints. The benefit is statelessness — any instance can validate the token without shared session storage, which scales horizontally."

Q: How would you handle a downstream service being slow or down?

Apply resilience patterns: a **timeout** so I don't block indefinitely, a **retry** with backoff for transient failures, and a **circuit breaker** (e.g., Resilience4j) that trips after repeated failures to avoid hammering a dead service, plus a **fallback** for graceful degradation. For non-critical calls, make them **asynchronous** via messaging (Kafka) to decouple.

Q: REST synchronous calls vs Kafka messaging — when do you use each?

Synchronous REST when the caller needs an immediate response and the operation is request-scoped (e.g., fetch customer details). Asynchronous messaging (Kafka) when you want decoupling, buffering, and resilience — events that multiple consumers care about, or work that can happen eventually (e.g., my TCS work where order events flowed through Kafka to downstream services). Messaging trades immediate consistency for scalability and resilience.

4.3 Microservices Pitfalls to Mention (Shows Maturity)

- **Distributed transactions** — avoid 2PC; use the Saga pattern with compensating actions
- **Data consistency** — eventual consistency via events; idempotent consumers
- **Service discovery & config** — externalized config, no hardcoded endpoints
- **Cascading failures** — circuit breakers, bulkheads, timeouts
- **Observability** — correlation IDs / distributed tracing to debug across services
- **Versioning** — backward-compatible API changes, consumer-driven contracts

4.4 Spring Practice Checklist

- ☐ Draw the Spring Security filter chain for JWT from memory
- ☐ Explain `@Transactional` self-invocation pitfall out loud

- ☐ Write a `JOIN FETCH` query to solve an N+1
 - ☐ Sketch how Resilience4j circuit breaker states transition (closed→open→half-open)
 - ☐ Be ready to whiteboard a clean REST API (resources, verbs, status codes, versioning)
-

5. System Design

This is your #1 priority area. At 5.5 YOE for product companies, system design rounds carry the most weight and are where service-firm candidates most often stumble. The good news: your billing + Kafka + microservices background gives you real material to draw on. We just need to make you *fluent* in the framework.

5.1 The Universal Framework (Memorize This)

Use this exact sequence in every design round. Spend ~45 min total, roughly:

1. REQUIREMENTS (5 min)
 - └ Functional: what must it DO?
 - └ Non-functional: scale, latency, availability, consistency?
 - └ Clarify scope – narrow it. Confirm with interviewer.
2. ESTIMATES (3 min)
 - └ Users (DAU), requests/sec (QPS), read:write ratio
 - └ Data size / storage growth
 - └ Bandwidth – sanity-check, don't over-invest
3. API DESIGN (5 min)
 - └ Key endpoints: signatures, request/response
4. DATA MODEL (5 min)
 - └ Core entities & relationships
 - └ SQL vs NoSQL – justify the choice
5. HIGH-LEVEL DESIGN (10 min)
 - └ Draw boxes: clients → LB → services → DB/cache/queue
 - └ Walk through the main flow end to end
6. DEEP DIVE (10 min)
 - └ Pick 1-2 components, go deep (the interviewer will steer)
7. SCALE & BOTTLENECKS (7 min)
 - └ Caching, replication, sharding, partitioning
 - └ Single points of failure
 - └ Trade-offs you're making (always name them)

The #1 thing interviewers want: that you **drive the conversation, state trade-offs explicitly**, and **don't pretend there's one right answer**. Saying *"I'll use X here, trading off Y for Z — does that align with the priorities?"* scores huge.

5.2 Core Building Blocks (Have These Ready)

Concept	One-Line Mental Model	When to Reach For It
Load Balancer	Distributes traffic across servers	Always, for horizontal scale
Cache (Redis)	Fast in-memory store	Read-heavy, repeated reads
CDN	Edge cache for static content	Geo-distributed static assets (Akamai's bread & butter!)
SQL	Strong consistency, joins, transactions	Financial, relational data (your billing world)
NoSQL (key-value/doc)	Scale, flexible schema, fast lookups	High write volume, simple access patterns
Message Queue (Kafka)	Async, decoupling, buffering	Event streams, decoupled processing (your TCS world)
Replication	Copies of data for availability/reads	HA + read scaling
Sharding/Partitioning	Split data across nodes	When one DB can't hold/serve it all
Consistent Hashing	Distribute keys, minimize reshuffling	Sharding, caching layers
Rate Limiter	Cap requests per client	API protection

CAP theorem (say it confidently): In a network partition, you choose Consistency or Availability. Banking/billing → CP. Social feed / cart → AP (eventual consistency). Most real systems tune per-feature.

5.3 Eight System Design Problems (Tailored to You)

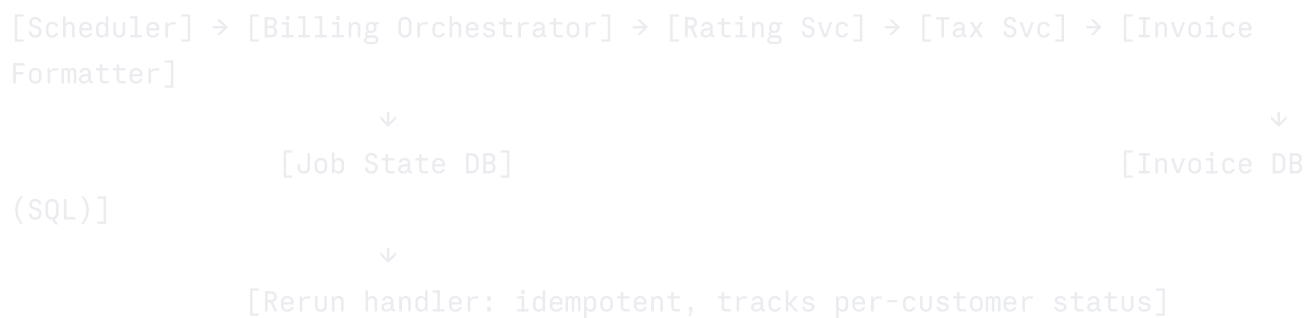
Ordered easiest → hardest. The first three lean on YOUR domain experience — start there for confidence.

● Design 1 — A Billing / Invoicing System (*Your home turf — start here*)

Why for you: You can speak from real MBS experience. This builds confidence.

Key discussion points:

- **Pipeline stages:** rating → billing → taxing → formatting (you know this!)
- **Batch processing:** process customers in chunks (you did 1,000-record chunks) — discuss idempotency so a rerun doesn't double-bill
- **Bill Rerun:** how do you safely reprocess? (your actual story) — idempotency keys, clear success/partial/fail states
- **Data store:** SQL (Oracle/Postgres) — financial data needs ACID, transactions
- **Scale:** 7,000 customers is small; discuss how you'd scale to millions (partition by customer, parallelize batch workers)



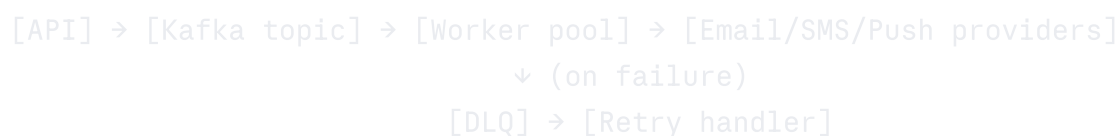
Trade-off to name: Strong consistency (SQL) over availability — you can't have billing errors.

● Design 2 — A Notification Service

Functional: Send email/SMS/push to users; support templates; handle retries; dedupe.

Key points:

- **Async via Kafka:** producer accepts notification requests → topic → consumer workers send them (your Kafka experience!)
- **Channel abstraction:** strategy pattern per channel (email/SMS/push)
- **Retries + DLQ:** failed sends go to a dead-letter queue
- **Idempotency:** dedupe by notification ID so retries don't double-send
- **Rate limiting:** respect provider limits



Trade-off: At-least-once delivery (simpler, needs idempotent consumers) vs exactly-once (complex).

● Design 3 — A URL Shortener (Classic Warm-Up)

Functional: Long URL → short code; redirect; analytics optional.

Key points:

- **Encoding:** base62 of an auto-increment ID, or hash + collision check
- **Read-heavy:** 100:1 read:write → cache hot URLs in Redis
- **Storage:** key-value (code → long URL); NoSQL fine here
- **Scale:** cache layer absorbs most reads; DB for source of truth

Trade-off: Counter-based codes (sequential, predictable) vs hash-based (random, collision handling).

● Design 4 — A Rate Limiter

Functional: Limit each client to N requests per window.

Key algorithms (know these):

- **Token bucket:** refill tokens at fixed rate; allow if token available (smooth, allows bursts)
- **Sliding window log/counter:** more accurate, more memory
- **Fixed window:** simplest, but boundary bursts

Distributed: Store counters in **Redis** (atomic INCR + TTL). Discuss the race condition and how Redis atomic ops / Lua scripts solve it.

Trade-off: Accuracy vs memory/complexity.

● Design 5 — An Inventory / Order Management System (*Your TCS domain*)

Why for you: Maps to your retail inventory work.

Functional: Track stock across warehouses, process orders, prevent overselling.

Key points:

- **Concurrency:** two orders for the last item — how do you prevent overselling? (Optimistic locking with version column, or atomic decrement)
- **Event-driven:** order placed → Kafka event → inventory update, fulfillment, notification (your actual architecture!)
- **Consistency:** inventory count must be accurate → SQL with row locking, or a reservation pattern
- **Scale:** partition by warehouse / SKU range

Trade-off: Strong consistency on stock counts (lock contention) vs eventual (risk of oversell, compensate later).

● Design 6 — A News Feed / Activity Feed

Functional: Users follow others; see a feed of recent activity.

Key points:

- **Fan-out on write** (push to followers' feeds — fast reads, expensive for celebrities) vs **fan-out on read** (pull at read time — cheap writes, slow reads). **Hybrid** for high-follower accounts.
- **Storage:** feed cache in Redis; posts in NoSQL
- **Pagination:** cursor-based, not offset

Trade-off: Write amplification vs read latency — the classic feed trade-off.

● Design 7 — A Distributed Cache (Akamai-Relevant)

Functional: Key-value cache across many nodes; high availability; eviction.

Key points:

- **Consistent hashing:** distribute keys, minimize reshuffle on node add/remove
- **Replication:** each key on N nodes for availability
- **Eviction:** LRU / LFU
- **Consistency:** write-through vs write-back vs write-around
- **Hot keys:** replicate hot keys to avoid a single node melting

Trade-off: Consistency vs availability per CAP; staleness vs write cost.

● Design 8 — A Payment Processing System (Fintech, Your RECV-Adjacent)

Functional: Process payments, ensure no double-charge, handle failures, reconcile.

Key points:

- **Idempotency keys:** client sends a unique key; server dedupes → no double-charge (critical!)
- **State machine:** payment states (initiated → authorized → captured → settled / failed)
- **Saga pattern:** distributed transaction across payment, ledger, notification with compensating actions
- **Consistency:** financial → strong consistency, audit log, exactly-once semantics
- **Reconciliation:** periodic job to catch mismatches

Trade-off: Strict correctness/consistency over latency — money can't be wrong.

5.4 System Design Practice Plan

- ☐ Week 1: Designs 1, 2, 3 (your comfort zone — build confidence)
- ☐ Week 2: Designs 4, 5 (rate limiter, inventory)

- ☐ Week 3: Designs 6, 7 (feed, distributed cache)
- ☐ Week 4: Design 8 + re-do Design 1 under time pressure
- ☐ For each: produce a 1-page diagram + written trade-offs
- ☐ Do at least 2 *out loud*, timed at 45 min

Top resources:

-  *System Design Interview* Vol 1 & 2 — Alex Xu (paid, the standard)
 -  github.com/donnemartin/system-design-primer (free, comprehensive)
 -  ByteByteGo YouTube channel (free, visual)
 -  Hello Interview (free system design walkthroughs, very current)
-

6. Algorithms & Coding (DSA)

Your reality: This is your other  priority. Product companies (especially Walmart, Samsung) use DSA screens. You don't need competitive-programming wizardry — you need **fluency in core patterns** and the ability to communicate while coding. Target: solve Medium problems in ~30 min with clean code.

6.1 Pattern-First Strategy (Not Random Grinding)

Don't grind 500 random problems. Learn **patterns** — most interview questions are variations of ~15 patterns.

Pattern	Frequency	Example Problems
Two Pointers	🔴 Very High	Two Sum II, Container With Most Water, 3Sum
Sliding Window	🔴 Very High	Longest Substring w/o Repeat, Max Subarray
Hashing / Maps	🔴 Very High	Two Sum, Group Anagrams, Subarray Sum
Binary Search	🔴 High	Search Rotated Array, Find Min, Koko Bananas
BFS / DFS (Trees & Graphs)	🔴 Very High	Level Order, Number of Islands, Course Schedule
Linked List	🟡 High	Reverse List, Detect Cycle, Merge K Lists
Stack / Queue	🟡 High	Valid Parentheses, Daily Temperatures, Min Stack
Heap / Priority Queue	🟡 High	Top K Elements, Merge K Lists, Median Stream
Dynamic Programming	🟡 Medium	Climbing Stairs, Coin Change, House Robber, LIS
Backtracking	🟢 Medium	Subsets, Permutations, Combination Sum
Intervals	🟡 Medium	Merge Intervals, Meeting Rooms
Greedy	🟢 Medium	Jump Game, Gas Station
Matrix	🟢 Medium	Rotate Image, Spiral, Set Matrix Zeroes
Trie	🟢 Low-Med	Implement Trie, Word Search II
Bit Manipulation	🟢 Low	Single Number, Counting Bits

6.2 Curated 60-Problem List (Blind-75-Style, Prioritized)

If you do **only these 60**, well, with full understanding, you'll handle most screens. ✅ = must-do.

Arrays & Hashing (do first)

- ✅ Two Sum (Easy)
- ✅ Best Time to Buy/Sell Stock (Easy)
- ✅ Contains Duplicate (Easy)
- ✅ Product of Array Except Self (Medium)
- ✅ Group Anagrams (Medium)
- ✅ Top K Frequent Elements (Medium)
- Maximum Subarray (Medium)
- Maximum Product Subarray (Medium)

Two Pointers 9. Valid Palindrome (Easy) 10. 3Sum (Medium) 11. Container With Most Water (Medium)

Sliding Window 12. Longest Substring Without Repeating Characters (Medium) 13. Longest Repeating Character Replacement (Medium) 14. Minimum Window Substring (Hard)

Binary Search 15. Binary Search (Easy) 16. Search in Rotated Sorted Array (Medium) 17. Find Minimum in Rotated Sorted Array (Medium) 18. Koko Eating Bananas (Medium)

Linked List 19. Reverse Linked List (Easy) 20. Merge Two Sorted Lists (Easy) 21. Linked List Cycle (Easy) 22. Reorder List (Medium) 23. Remove Nth Node From End (Medium) 24. Merge K Sorted Lists (Hard)

Stack 25. Valid Parentheses (Easy) 26. Min Stack (Medium) 27. Daily Temperatures (Medium) 28. Evaluate Reverse Polish Notation (Medium)

Trees 29. Invert Binary Tree (Easy) 30. Maximum Depth of Binary Tree (Easy) 31. Same Tree (Easy) 32. Binary Tree Level Order Traversal (Medium) 33. Validate Binary Search Tree (Medium) 34. Lowest Common Ancestor of BST (Medium) 35. Kth Smallest Element in BST (Medium) 36. Construct Tree from Preorder/Inorder (Medium)

Heap / Priority Queue 37. Kth Largest Element in Array (Medium) 38. Find Median from Data Stream (Hard) 39. Task Scheduler (Medium)

Graphs (high value) 40. Number of Islands (Medium) 41. Clone Graph (Medium) 42. Course Schedule (Medium) — topological sort 43. Pacific Atlantic Water Flow (Medium) 44. Rotting Oranges (Medium) — BFS

Backtracking 45. Subsets (Medium) 46. Combination Sum (Medium) 47. Permutations (Medium) 48. Word Search (Medium)

Dynamic Programming 49. Climbing Stairs (Easy) 50. Coin Change (Medium) 51. House Robber (Medium) 52. Longest Increasing Subsequence (Medium) 53. Unique Paths (Medium) 54. Word Break (Medium) 55. Longest Common Subsequence (Medium)

Intervals 56. Merge Intervals (Medium) 57. Insert Interval (Medium) 58. Non-overlapping Intervals (Medium)

Greedy / Misc 59. Jump Game (Medium) 60. Valid Anagram (Easy)

6.3 Problem-Solving Communication Template

Interviewers grade *how you think*, not just the answer. Narrate using this:

1. CLARIFY → "Can the array have negatives? Duplicates? Is it sorted?"
2. EXAMPLE → Walk a small input by hand
3. BRUTE FORCE → State the naive $O(n^2)$ approach first — shows you can start
4. OPTIMIZE → "I can use a hashmap to bring this to $O(n)$..."
5. CODE → Write clean code, talk as you go
6. TEST → Walk through with your example + edge cases (empty, single, null)
7. COMPLEXITY → State time & space Big-O

Never code in silence. Even if stuck: "I'm thinking about whether a two-pointer works here because the array is sorted..."

6.4 Sample Worked Problem (Java)

Two Sum — the template-setter:

```
java

// Clarify: exactly one solution? Can't reuse same element? Return indices.
public int[] twoSum(int[] nums, int target) {
    // Map value -> index. One pass: for each num, check if its complement seen
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (seen.containsKey(complement)) {
            return new int[]{ seen.get(complement), i };
        }
        seen.put(nums[i], i);
    }
    return new int[]{}; // no solution
}

// Time:  $O(n)$  — single pass. Space:  $O(n)$  — the map.
// Brute force would be  $O(n^2)$  nested loops; the map trades space for time.
```

6.5 DSA Practice Checklist

- ☐ Solve 3–4 problems daily (mix of patterns, mostly Medium)
- ☐ Re-solve any problem you couldn't finish, 2 days later
- ☐ Time yourself: Medium in ≤ 30 min
- ☐ Always state Big-O at the end
- ☐ Keep a "missed problems" log and revisit weekly

Top resources:

- [NeetCode.io](#) — the Blind 75 / NeetCode 150 with video solutions (best free resource)

-  LeetCode (filter by company tag: Walmart, Samsung)
-  *Grokking the Coding Interview* (paid, pattern-based — great for your style)

7. Databases, DevOps & Cloud

Your reality: You've used Oracle, Postgres, MySQL, MongoDB, DynamoDB, plus Docker/K8s/Jenkins/GitHub Actions and AWS S3/DynamoDB. Solid breadth. Focus on articulating SQL depth and the DevOps concepts you've touched.

7.1 SQL & Databases

Topic	Priority	Expected Question
Indexing (B-tree, when to use)	●	"How would you speed up a slow query?" (your Bill Preview story!)
Joins, GROUP BY, window functions	●	Live SQL writing
Transactions & ACID, isolation levels	●	"Explain isolation levels / phantom reads"
Normalization vs denormalization	●	"When would you denormalize?"
SQL vs NoSQL trade-offs	●	"When NoSQL over SQL?"
Optimistic vs pessimistic locking	●	"How prevent lost updates?" (inventory!)

Q: How do you debug a slow query? (*You lived this.*)

"Start with `EXPLAIN`/`EXPLAIN ANALYZE` to see the execution plan — look for full table scans where an index should be used. Check if the right indexes exist on filter/join columns. Look for over-fetching (selecting unneeded columns/rows) and N+1 patterns from the ORM. In my Bill Preview case, the JPQL was doing inefficient joins and over-fetching; I refactored it to filter earlier and project only what was needed."

Q: SQL vs NoSQL — when which?

SQL for relational, transactional, consistency-critical data with complex queries — like billing and payments. NoSQL when you need horizontal scale, flexible schema, and simple access patterns — like a key-value store for sessions, or a document store for catalog data. I've used DynamoDB for job-state tracking at TCS where I needed fast key lookups at scale, and Oracle/Postgres for transactional billing data.

Q: Isolation levels?

Read Uncommitted (dirty reads) → Read Committed (no dirty reads) → Repeatable Read (no non-repeatable reads) → Serializable (no phantoms, full isolation). Higher isolation = more correctness, less concurrency. Most OLTP systems use Read Committed or Repeatable Read.

7.2 DevOps & Cloud

Topic	Priority	Expected Question
Docker (images, layers, containers)	●	"What's the difference between an image and a container?"
Kubernetes (pods, deployments, services)	●	"How does a rolling deployment work?"
CI/CD pipelines	●	"Walk me through your pipeline" (you have GitHub Actions + Jenkins)
AWS core (S3, EC2, DynamoDB, IAM)	●	"How would you store/serve files?"
Monitoring / logging	●	"How do you know a service is healthy?"

Q: Docker image vs container?

An image is an immutable template (layers of filesystem + metadata). A container is a running instance of an image with its own writable layer. One image → many containers.

Q: Walk me through your CI/CD pipeline. (Your real experience.)

"On a push/PR, the pipeline builds the code, runs unit tests (JUnit/Mockito) and quality gates (SonarQube, coverage via JaCoCo), and on success builds a Docker image and deploys to the target environment on Kubernetes. We promoted through DEV → TEST → PREPROD → PROD. I used GitHub Actions for the ATC application and Jenkins elsewhere."

Q: How does a Kubernetes rolling deployment avoid downtime?

K8s spins up new-version pods while keeping old ones serving, waits for readiness probes to pass, then gradually shifts traffic and terminates old pods. If new pods fail health checks, the rollout halts. This gives zero-downtime deploys and easy rollback.

7.3 DB/DevOps Checklist

- ☐ Write 5 SQL queries with joins + GROUP BY + a window function

- ☐ Explain B-tree indexing and when an index *hurts* (write-heavy)
 - ☐ Explain optimistic locking with a version column (ties to inventory design)
 - ☐ Whiteboard your CI/CD pipeline end to end
 - ☐ Explain Docker layers and image caching
-

8. Mock Interview Plan & Feedback Framework

8.1 Structured Mock Session Format

Run **2 mocks per week** (with me here, or a peer). Each ~60 min:

Phase	Time	Focus
Warm-up	5 min	"Tell me about yourself" + 1 resume follow-up
Technical / Coding	25 min	1 DSA problem, narrated aloud
System Design (<i>alternate weeks</i>)	25 min	1 design from Section 5
Behavioral	15 min	2 STAR stories, probed
Feedback	15 min	Scored + specific fixes

8.2 Feedback Scorecard (Self-Assess After Each)

Dimension	1-5	Notes
Communication / clarity		Did I narrate my thinking?
Problem-solving approach		Brute force → optimize?
Code quality		Clean, correct, tested?
Trade-off articulation		Did I name trade-offs?
Behavioral structure		STAR followed? Quantified?
Time management		Finished in time?

Rule: After every mock, write down the **top 2 things to fix**. Bring them to the next session.

8.3 How to Practice With Me

Just say, e.g.:

- "Give me a mock DSA round on graphs"
- "Mock system design: design a notification service, grill me"
- "Behavioral round — ask me 5 questions and score me"
- "Ask me Spring Boot questions like a Walmart interviewer"

9. Four-Week Study Schedule

Assumption: ~2.5–3 hrs/day on weekdays, ~5 hrs/day weekends (~20 hrs/week). Adjust to your reality. Front-loads your two weak areas (DSA + System Design) while keeping Java/Spring warm.

Week 1 — Foundations + Confidence

Day	DSA (60–75m)	Theory (45–60m)	Other (30m)
Mon	Arrays/Hashing #1–4	Core Java: Collections	Polish "Tell me about yourself"
Tue	Arrays/Hashing #5–8	Core Java: Concurrency basics	STAR stories 1–3 aloud
Wed	Two Pointers #9–11	Spring: DI, beans, @Transactional	—
Thu	Sliding Window #12–13	Spring: JWT + Security (your story)	STAR 4–6
Fri	Binary Search #15–17	System Design framework (5.1)	—
Sat	Review week's misses	System Design 1 (Billing)	Mock: behavioral
Sun	Linked List #19–21	System Design 2 (Notification)	Rest / review

Milestone: 20+ problems done; framework memorized; 3 anchor stories smooth.

Week 2 — Depth + Breadth

Day	DSA	Theory	Other
Mon	Stack #25–27	Java: JVM/GC, equals/hashCode	STAR 7–9
Tue	Trees #29–32	Spring: N+1, JPA tuning	—
Wed	Trees #33–36	DB: indexing, slow query (your story)	—
Thu	Heap #37–39	DB: SQL practice (joins, windows)	STAR 10–12

Day	DSA	Theory	Other
Fri	Graphs #40-42	System Design 3 (URL) + 4 (Rate limiter)	—
Sat	Review + re-solve misses	System Design 5 (Inventory)	Mock: DSA round
Sun	Backtracking #45-47	DevOps: Docker, CI/CD, K8s	Rest

Milestone: 40+ problems; 5 designs done; SQL + DevOps Q&A solid.

Week 3 — Hard Topics + Integration

Day	DSA	Theory	Other
Mon	DP #49-51	Java 17/21 features	Refine weak STAR stories
Tue	DP #52-55	Microservices: resilience, saga	—
Wed	Graphs #43-44	System Design 6 (Feed)	—
Thu	Intervals #56-58	System Design 7 (Distributed cache)	—
Fri	Mixed review #22-24, 33	Review all Spring/Java Q&A	—
Sat	Timed Medium ×3	System Design 8 (Payments)	Mock: System design
Sun	Re-solve hardest misses	Full Q&A self-quiz	Rest

Milestone: 55+ problems; all 8 designs done; can discuss modern Java.

Week 4 — Polish + Simulate

Day	Focus
Mon	Full mock interview (coding + behavioral) + fix top 2 gaps
Tue	Re-do System Design 1 & 8 under 45-min timer
Wed	Full mock interview (system design + behavioral)
Thu	Company-specific prep (LeetCode company tags; revisit values cheat sheet)
Fri	Re-solve your 10 most-missed problems; final Q&A review
Sat	Light review, rest, logistics (LinkedIn, applications)
Sun	Rest. Confidence consolidation. No cramming.

Milestone: Interview-ready across all dimensions.

9.1 Schedule Adjustments

If you have only 2 weeks: Do Week 1 + Week 3. Drop DP depth (keep Coin Change, House Robber, Climbing Stairs). Do 4 system designs (1, 2, 5, 8). Prioritize mocks.

If you have 6-8 weeks: Stretch each week, add NeetCode 150 (beyond the 60), do all 8 designs twice, run 3 mocks/week, add a second pass on concurrency and JVM internals.

If working full-time / limited hours: Halve daily DSA to 2 problems, protect the weekend system-design blocks (non-negotiable), and prioritize mocks over passive reading.

9.2 Master Progress Tracker

- ☐ Week 1 complete — 20 problems, framework, 3 stories, 2 designs
- ☐ Week 2 complete — 40 problems, 5 designs, SQL/DevOps solid
- ☐ Week 3 complete — 55 problems, 8 designs, modern Java
- ☐ Week 4 complete — 60 problems, 2 full mocks, company prep
- ☐ LinkedIn URL added to resume + AZ-900 added to profile
- ☐ Applied to first batch of target companies

10. Honest Gaps & Quick Wins Summary

Gap	Honest Risk	Quick Win
System design fluency	Highest-weighted round	The framework (5.1) + your 3 domain designs (1, 2, 5) — you already know the material
DSA under time	Screen-out risk	Pattern-first (6.1) + NeetCode 60 — don't grind randomly
Java 17/21	Minor probe	2-3 hrs on the 5 features in 3.3
No formal "design ownership"	Seniority question	Reframe Bill Rerun + MCA + ATC as design <i>decisions</i> you made
Concurrency depth	Common Java probe	Section 3.2 questions + write the 3-way counter
"Failure" / "conflict" stories	You don't have these ready	Bring real examples next session — we'll build them

11. Clarifying Questions for the Next Phase

To tailor the next session, tell me:

1. **Timeline** — How many weeks until your first interview? (Sets which schedule variant.)
2. **Hours/day** — Realistically, how much time can you commit daily?
3. **Weakest area by feel** — Where do you feel *least* confident: DSA, system design, or behavioral?
4. **DSA baseline** — Have you done LeetCode before? Roughly how many problems?
5. **First mock** — Which round do you want to mock first: a DSA problem, a system design, or a behavioral grilling?
6. **Failure & conflict stories** — Can you recall a real small mistake at work and a real technical disagreement? (I'll help you frame both safely.)

Pick any one and we'll start. My recommendation: **begin with a system design mock on the Billing System (Design 1)** — it's your home turf, it'll build confidence fast, and it's your highest-priority round.

Built for Abubaker Siddique · Java Backend Engineer · Targeting Walmart, Target, Samsung, Akamai